云法智办 · 架构现状说明

代码架构 / 模型架构 / 启动脚本位置 —— 一页对齐「东西在哪、怎么跑起来」

2026-10-27 | 仓库 D:\Projects\judicial_app（分支 app） | 生产 10.128.85.133 · 银河麒麟 V10 · ARM64 | 内部汇报用

1 个后端
FastAPI 单体，内部分 4 层

2 套 Compose
API+DB / 模型池，分开编排

8 张卡
310P，当前实际只占 5 张

1 套模型
35B 同时接管文本/视觉/长文

0 先讲四句话

会上如果只记四件事，记这四件。

问题	答案
系统跑在哪	生产服务器 `10.128.85.133` 上两个 Docker 世界：**业务编排**（API + PostgreSQL）和 **模型池**（35B / BGE）；客户端是装在办公电脑上的桌面程序，默认连 `http://10.128.85.133:8000`。
代码怎么分	后端 `server/app` 按 接口层 → 应用层 → 领域层 → 基础设施层 四层切，横切 `core`；桌面端 `desktop` 按 15 个功能页切。业务新增只动应用层 + 接口层，不碰技术实现。
模型怎么用	只有一套在线模型：35B（`:8010`，占 2 卡），文本、视觉、长文三条链路全指向它；向量化走 bge-m3（`:8006`）。PPT 的 9B 栈已退出正式链路，但容器还在占卡。
脚本在哪	生产权威脚本全在 `deploy/`：`deploy_latest.sh` 负责部署，`production.docker-compose.yml` + `model-pool/docker-compose.yml` 负责编排。**仓库根目录那批 `start_all.sh` / `start_services.sh` 是 14B 裸进程时代的遗留，已不可用。**

1 运行时拓扑：谁在跑、跑在哪台机器

这张图回答「点一下按钮，请求经过了哪几个进程」。注意模型不在 API 容器里，是宿主机上独立的一套 Docker。



图 1 | 部署视图。绿色区是模型池，和业务编排是两套独立 compose，互不 restart 牵连。

会上要强调的一点：业务容器里没有模型。模型是宿主机上单独跑的 vllm 服务，API 只通过 HTTP 调它。所以「重启后端」不会影响模型，「模型挂了」也不会让后端进程退出——只会让 AI 功能超时。

2 代码架构：四层，一条依赖方向

后端是单体，但内部严格分层。判断一段代码该写在哪，看它「碰什么」：碰 HTTP 去 interfaces，碰业务流程去 application，碰专业规则去 domain，碰数据库/模型/文件去 infrastructure。

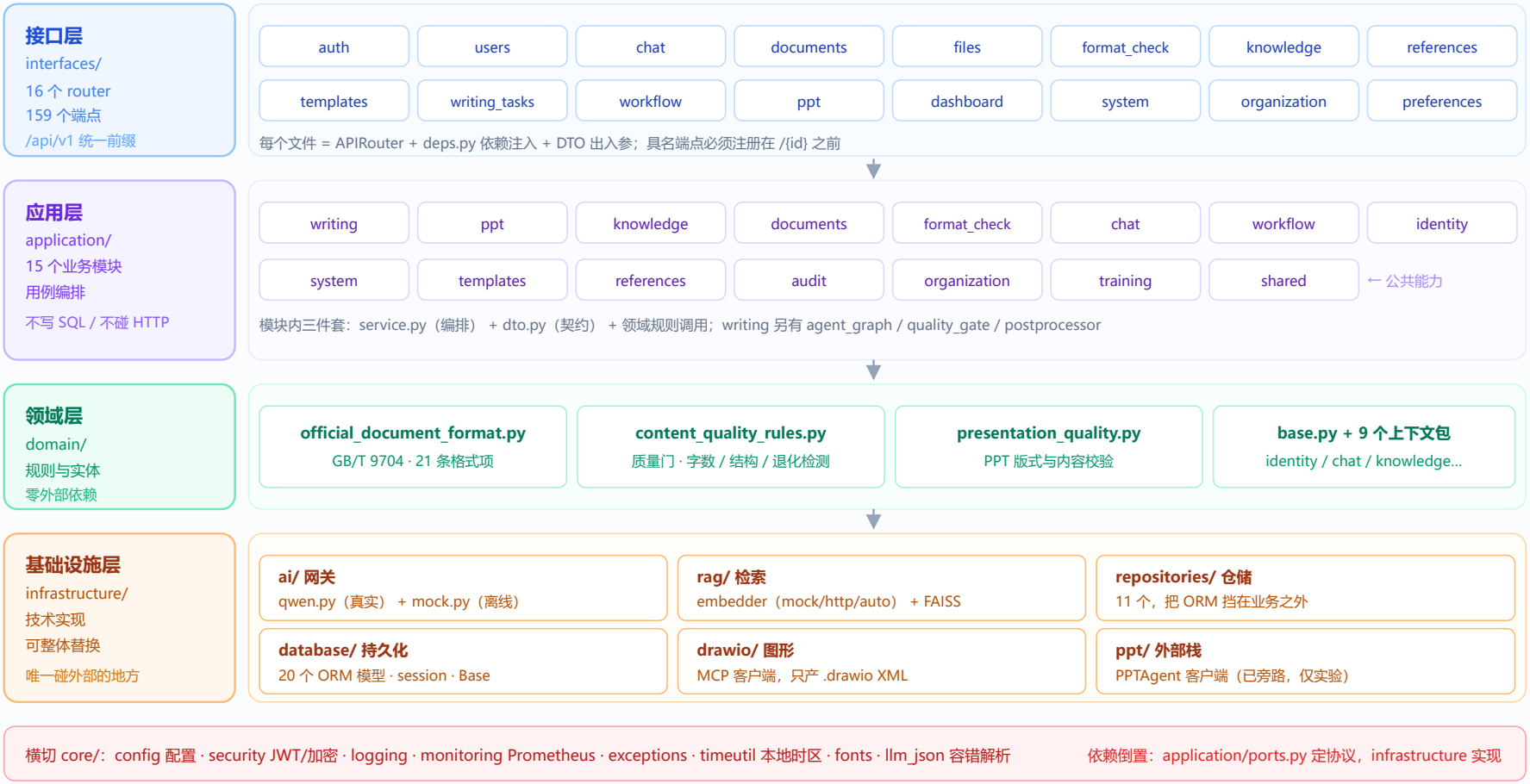


图 2 | 代码分层。虚线箭头表示依赖倒置：基础设施实现应用层定义的端口，领域层谁都不依赖。

> **一句话判断法：**新功能加在哪一层，看它最外侧碰什么。要新增一个「公文格式检查项」——规则进 `domain/`，流程进 `application/format_check/`，开接口进 `interfaces/format_check.py`，三处都不用改别的模块。

3 模型架构：一套 35B，四套网关，8 张卡

这里最容易讲乱。抓住一句：**网关有四个，模型只有一个。**四套网关只是四种参数策略（超时、温度、上下文、是否带图），终点都是同一个 `:8010`。



图 3 | 模型架构。左侧四套网关 + 一条 RAG 链路，中间是实际的服务实例，右侧是卡位占用。

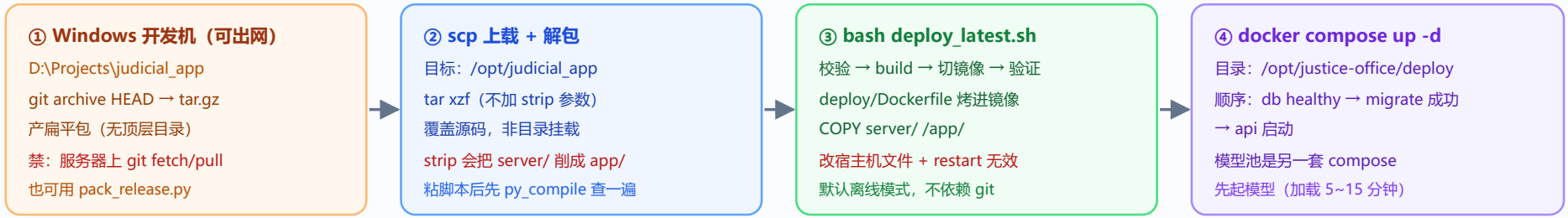
模型相关环境变量（生产 .env）

变量	当前应填	作用
`MODEL_SERVICE_URL`	`http://10.128.85.133:8010/v1`	主模型地址。**必须是宿主 LAN IP**，不能填 127.0.0.1

变量	当前应填	作用
`MODEL_NAME`	`Qwen3.6-35B-A3B-W8A8`	served-model-name，须与 vllm 启动参数一致
`VISION_MODEL_URL`	同上（:8010）	留空则退回纯 OCR 文本通道，多模态失效
`LONG_FORM_MODEL_URLS`	同上（:8010）	留空仍能生成，但无显式关思考，长文更慢更飘
`EMBEDDING_PROVIDER`	`http`	走远程 bge，避免容器内 CPU 向量化卡 8 秒超时
`PPT_AGENT_ENABLED`	`false`	关掉外部 PPTAgent，走 35B + 本地排版引擎
`DRAWIO_MCP_URL`	留空或 `http://drawio-mcp:3001`	留空即不启用，PPT 关系图走本地渲染
`MODEL_MAX_CONCURRENCY`	`4`	网关侧并发闸门，超了排队而不是打爆模型

4 启动与部署：四步，位置写死

生产环境严禁出网，所以没有 `git pull`。源码在 Windows 端打包、scp 上去、服务器上解包再 build。下面这条链是唯一正确路径。



上线后必须核对（否则容易「假成功」）

docker inspect justice-api --format '{{.Config.Image}}' ← 确认镜像真的切过去了

curl -s http://127.0.0.1:8000/health · tail -f /data/justice/logs/server.log · docker compose ps 看三容器状态

⚠ 模型池与业务是两套 compose：只要 API 能通过 HTTP 摸到 :8010 就行，谁先起都可以；但模型加载慢，演示前务必预留 15 分钟。

⚠ 纯空气闸（完全无外网）时：出网机 build → docker save → scp → docker load → 在 .env 设 SERVER_IMAGE → up -d，现场不 build。

⚠ preflight-production.sh 里的模型端口/路径是硬编码的历史布局（:8003/:8002/:8004），改模型布局时要同步改它。

图 4 | 发布链路。注意第 3 步是「构建」不是「重启」，这是历史上反复踩的坑。

5 东西到底放在哪：目录与脚本清单

这一节是会上最容易被追问的。左边是仓库（开发机），右边是服务器上的绝对路径，两者不要混。

5.1 仓库结构（D:\Projects\judicial_app，分支 app）

```
D://Projects//judicial_app//                唯一源码仓库 · 开发机（Windows）
├─ server\                                  后端 · Python 3.11 + FastAPI
│  ├─ app\main.py                           应用入口：create_app / lifespan / 16 路由装配
│  ├─ app\interfaces\                       HTTP 路由层（16 个 router）
│  ├─ app\application\                      用例编排（15 个业务模块）
│  ├─ app\domain\                           领域规则（GB/T 9704、质量规则）
│  ├─ app\infrastructure\                   AI 网关 / RAG / 仓储 / ORM / drawio
│  ├─ app\core\                             配置 / 安全 / 日志 / 监控
│  └─ app\migrations\                       Alembic 迁移（19 个版本文件）
├─ desktop\                                 桌面客户端 · Compose Desktop（Kotlin）
│  ├─ src\jvmMain\kotlin\...\features\      15 个功能页（writing / ppt / knowledge…）
│  └─ installer\                            安装包产物（EXE / deb）
├─ deploy\                                  ★ 部署资产 · 生产权威来源
│  ├─ production.docker-compose.yml         生产 API + DB 编排（服务器同名文件以此为准）
│  ├─ model-pool\docker-compose.yml         模型池编排（35B / bge / 14B 回滚 / 试用）
│  ├─ deploy_latest.sh                      覆盖部署脚本（校验 → 构建 → 切流量）
│  ├─ Dockerfile                            后端镜像（COPY server/ /app/）
│  ├─ docker-compose.yml                    开发 / 单机用（SQLite），非生产
│  └─ backup-production.sh                  生产备份（独立 justice-db 容器）
├─ start-backend.bat / .ps1                 本地开发起后端（Windows）
├─ desktop.sh                               在图形桌面起客户端（演示用）
├─ start_all.sh / start_services.sh         ✗ 遗留：vLLM 14B 裸进程时代（:8001）
├─ stop_services.sh / status.sh             ✗ 遗留：只管 8000/8001，与新架构不符
└─ start-tunnels.bat                        ✗ 过期：指向 192.168.68.51 实验机
```

5.2 服务器上的绝对路径（10.128.85.133）

路径	是什么	谁在用 / 怎么用
`/opt/justice-office/deploy/docker-compose.yml`	生产编排（API+DB）	真正的生产 compose，仓库里 `deploy/production.docker-compose.yml` 是它的版本化副本
`/opt/justice-office/deploy/.env`	生产环境变量	密码 / SECRET_KEY / 模型地址；模板 `production.env.example`
`/opt/justice-office/deploy/model-pool/docker-compose.yml`	模型池编排	`docker compose up -d` 起 35B + bge；`--profile rollback|trial` 起回滚/试用

路径	是什么	谁在用 / 怎么用
`/opt/judicial_app/`	源码落地目录	只作为 build 的上下文来源，改这里不重启不生效
`/data/justice/`	业务数据卷	uploads / ppt / generated / faiss / format-check，备份对象就是这个目录 + 数据库
`/data/justice/logs/server.log`	后端日志	排障第一现场；容器内 `DATA_DIR=/data/justice`
`/opt/justice-ppt/workspace`	PPT 外部栈工作区	只读挂到 API 容器 `/data/pptagent`；PPTAgent 下线后可清理

5.3 脚本清单与可用性（重点：哪些别再用了）

脚本 / 文件	状态	位置	说明
`deploy_latest.sh`	权威	`deploy/`	生产唯一部署入口。默认离线模式；`--check-only` 只校验；**不要传** `--build-arg`
`production.docker-compose.yml`	权威	`deploy/` → 服务器 `/opt/justice-office/deploy/`	起 justice-db / justice-migrate / justice-api 三个容器
`model-pool/docker-compose.yml`	权威	`deploy/model-pool/`	起 35B(:8010) + bge(:8006)；`profiles: rollback / trial` 控制回滚与试用模型
`app.migrations.migrator`	权威	容器内 `python -m`	迁移只由 `justice-migrate` 服务跑一次；**不要用** `docker exec alembic`
`backup-production.sh`	权威	`deploy/`	在宿主机执行，直连 justice-db 容器；API 容器内 psql 必失败
`deploy/docker-compose.yml`	仅开发	`deploy/`	单机 SQLite，带 `--build`；**不是生产编排**，别拿它上服务器
`start-backend.bat / .ps1`	仅开发	仓库根目录	Windows 本地起 uvicorn :8000，改代码即生效，用于调试
`desktop.sh`	仅演示	仓库根目录	需要在图形桌面（DISPLAY / WAYLAND）里跑，起 Compose Desktop 客户端
`start_all.sh` / `start_services.sh`	已废弃	仓库根目录	vLLM 起 Qwen2.5-14B 占 :8001 的裸进程方案，与 35B + Docker 现状完全不符
`stop_services.sh` / `status.sh`	已废弃	仓库根目录	只认 8000/8001；现在端口是 8000/8010/8006，跑它只会得到错误结论
`start-tunnels.bat`	已过期	仓库根目录	SSH 隧道指向 `192.168.68.51` 实验机（PPTAgent / llama.cpp），正式环境不用

> **建议在本次会上拍板：**把三个废弃脚本移进 `deploy/archive/` 并在 `README.md` 顶部列出「生产只用哪三个入口」。现在根目录 8 个脚本里只有 2 个还能用，交接时极易踩错。

6 端口总表：一个端口对一件事

现场排障时先看端口。这张表建议直接贴到运维文档里。

端口	服务	所在位置	状态 / 说明
`8000`	justice-api（FastAPI）	业务 compose	必开 客户端唯一入口，`/api/v1`、`/health`、`/docs`
`5432`	PostgreSQL 15	justice-db 容器内	必开 不发布到宿主机，只在 justice-net 内
`8010`	Qwen3.6-35B-A3B-W8A8	模型池 · davinci0,1	必开 文本 / 视觉 / 长文三链路共用
`8006`	bge-m3 向量化	模型池 · davinci2	必开 RAG 依赖；维度须与 FAISS 一致
`8003`	Qwen2.5-14B 回滚	模型池 · davinci7	按需 `--profile rollback`，默认不起
`8004`	DeepPresenter-9B	PPTAgent 栈 · davinci5,6	建议停 正式链路已旁路，停后可释放 2 卡
`7861`	PPTAgent Host（Gradio）	PPTAgent 栈	建议停 与 :8004 同进同退
`8011`	27B 试用	模型池 · davinci2,3	未启用 与 bge 争 davinci2，二选一
`8012`	flash-next 试用	模型池 · davinci4	未启用 权重未到位
`3001`	Draw.io MCP	独立容器	可选 只绑 127.0.0.1，MCP 无鉴权

7 业务模块对照：找功能先找这三个文件

会后如果有人问「XX 功能在哪」，按这张表三跳就能定位：路由 → 服务 → 规则。

模块	路由（interfaces/）	编排（application/）	核心能力
智能写作	`writing_tasks.py`	`writing/task_service.py`	分节生成、质量门、改写、定稿；25 个服务文件，最重的一块
PPT 生成	`ppt.py`	`ppt/service.py` `agent_job.py`	35B 出大纲 → 本地排版引擎 → 逐页预览；drawio 只画关系图
知识库	`knowledge.py`	`knowledge/rag_service.py`	上传/解析/OCR/切片/向量化/检索/重排，含网页导入
格式校验	`format_check.py`	`format_check/check_engine.py`	确定性规则 + 自动修正 + 35B 语义复核
AI 对话	`chat.py`	`chat/service.py`	SSE 流式、意图识别、附件解析、docx 导出、记忆
公文文档	`documents.py`	`documents/service.py`	文档 CRUD、排版、模板套用
流程	`workflow.py`	`workflow/service.py`	审批流转、内置流程数据
身份	`auth.py` `users.py`	`identity/service.py`	登录、JWT、首登改密、限流、短信

模块	路由（interfaces/）	编排（application/）	核心能力
组织	`organization.py`	`organization/service.py`	部门树、角色权限
模板/范文	`templates.py` `references.py`	`templates/` `references/`	内置模板库、范文检索与风格卡
个人偏好	`preferences.py`	`shared/user_preference.py`	助手名称、写作风格、行为开关
看板/系统	`dashboard.py` `system.py`	`system/service.py`	统计、健康检查、种子数据
审计	—（内部调用）	`audit/service.py`	操作留痕
培训	`training.py`	`training/service.py`	路由已注销（有意 404），仅保留代码与 LoRA 管线

8 需要会上拍板的六件事

这几项都是「现状已清楚、只差一个决定」，建议这次会上直接定下来，会后就能动。

#	事项	现状	建议决策 / 影响
1	PPTAgent 栈是否停用	DeepPresenter-9B（:8004）+ Host（:7861）容器仍在，占 **davinci5,6**；正式链路已改为 35B + 本地渲染	停容器 + 生产 .env 置空 `PPT_AGENT_URL`，**释放 2 张卡**
2	三项已修未部署	Embedder 路径修复、Draw.io 接入、PPT 本地逐页预览，代码已提交但**镜像未重建**	排一个发布窗口统一重建；否则功能在服务器上等于没有
3	缺失的 015 迁移文件	`alembic_version` 长度问题曾手工修库，仓库里没有对应迁移文件（014 不可改名）	补 `015_widen_alembic_version_num.py`，避免下次部署在别的库上再炸
4	格式校验改造排期	确定性 21 条已可用；短板是结构项不查、语义项给了问题不给改法、只能独立上传不能写作时实时	按 A 规则扩展 → B 语义可修正 → C 写作时实时 三步走
5	废弃脚本清理	根目录 8 个脚本里 3 个已失效，2 个仅开发用，交接易踩错	移入 `deploy/archive/`，README 顶部只留生产三入口
6	多智能体写作灰度	开关 `WRITING_AGENT_GRAPH_ENABLED` 默认 false，功能已就绪但未对外	定灰度比例（建议先 10% 复杂任务），观察质量与耗时

> **会议结束前的一句话总结：**代码分四层、模型就一套、部署走四个脚本——`deploy_latest.sh` 管发布，`production.docker-compose.yml` 管业务，`model-pool/docker-compose.yml` 管模型，`migrate` 服务管表结构。其余脚本都是历史遗留。